SentimentEdge

Phase 2 Report & Documentation

Architecture, Coordination, and Contribution

Phase 2 | Track A | Five-Agent Sequential Pipeline
Prototype for 1 week of data

April 2026

Table of Contents

Section 1 — Sequential Pipeline Justification
Section 2 — Agent Role Definitions
Section 3 — Coordination Logic, Orchestration, and Error Handling
Section 4 — AI Usage Disclosure
Section 5 — Team Contribution Update
Section 6 — Tools, Memory, and State Design
Section 7 — Evaluation Plan and Test Cases
Section 8 — Risk and Governance
Appendix — Prototype Evidence

Section 1: Sequential Pipeline Justification

SentimentEdge uses a five-agent sequential pipeline rather than a single-prompt approach or a two-agent alternative because each agent in the sequence addresses a distinct failure mode that the previous agent cannot handle, and that a combined agent would handle worse.

A single-prompt approach, feeding raw JSONL directly to Claude would fail immediately on the data quality problem. In the Phase 2 run, 64% of raw posts had deleted or removed bodies (`selftext = [removed]`), a structural property of r/wallstreetbets where moderators remove post content but preserve the thread. Sending these to Claude produces empty analyses that lower average confidence and consume API budget. The Filter Agent exists specifically to solve this before any LLM call is made: it accepts posts with body OR bundled comments, raising usable retention from under 5% to 18.2%. No prompt instruction can replicate this — it requires reading the comment dataset and joining it to posts at the data layer.

Collapsing the Sentiment Agent into the Aggregator would break the separation between reasoning and routing. The Aggregator's job is statistical: group by ticker, compute percentages, apply thresholds. Claude's job is linguistic: detect sarcasm, extract tickers with financial context, judge rumour credibility. In the Phase 2 run, 49.5% of posts were flagged as sarcastic — a rate driven by the April 2026 market downturn and r/wallstreetbets' characteristic response to losses through heavy irony. A rule-based aggregator cannot detect this. Conversely, asking Claude to also produce aggregated statistics per ticker inside a single prompt would require it to hold the full batch in context and perform group-by arithmetic — tasks it is poorly suited for compared to pandas.

The Output Agent exists separately because presentation logic changes independently of analysis logic. During Phase 2 development, the output format was restructured twice without touching the LLM prompt or the aggregation formula. A display change should not require re-running 283 API calls. The `--replay` flag formalises this separation: the Output Agent can be re-run against any saved run directory in seconds with no API cost.

The prototype evidence supports these boundaries directly. The highest-engagement post in the dataset (DPZ, engagement score 9,656.8) was a satirical post about Domino's pizza orders near the Pentagon. Claude correctly assigned it confidence 0.20 and market relevance 0.10. Without the Filter Agent's engagement-score-based batch selection and the Sentiment Agent's relevance judgment, this post would have dominated the ticker aggregate and produced a meaningless report. Each agent boundary exists because removing it would degrade a real, observed failure mode.

Section 2: Agent Role Definitions

SentimentEdge is built as a five-agent sequential pipeline. Each agent has a clearly bounded responsibility, receives structured input from the previous stage, applies its own logic independently, and passes a well-defined output to the next stage. The boundaries between agents are not arbitrary. Each one exists because collapsing it into an adjacent agent would either waste API budget, misclassify signal, or mix concerns that have fundamentally different failure modes.

Agent 1: Collector

Role

The Collector is the data ingestion layer. Its sole responsibility is to load raw posts and comments from JSONL files on disk into two pandas DataFrames, then pass them downstream unchanged. It performs

no filtering, no cleaning, and no analysis. The Collector's job is availability, not quality. Quality is the Filter Agent's job.

Inputs

Input	Source	Description
posts_file	config.py	Path to JSONL file of Reddit posts
comments_file	config.py	Path to JSONL file of Reddit comments

Decision Logic

The Collector reads each JSONL file line by line. Valid lines are parsed and appended to an in-memory list. Malformed lines are skipped with a logged warning rather than raising an exception. After loading, the Collector reports row counts to stdout and returns the two DataFrames. No filtering, deduplication, or schema enforcement happens here.

Failure Handling

If a JSONL file is missing, the Collector raises a FileNotFoundError with a diagnostic message. The run terminates immediately. If individual lines are malformed, they are skipped and counted. The pipeline continues as long as at least one valid post is loaded.

Outputs

Output	Key Columns	Count (Phase 2 run)
df_posts_raw	id, title, selftext, score, num_comments, flair, author, created_utc, permalink, stickied	1,559 posts
df_comments_raw	id, body, score, link_id, parent_id, author, created_utc	224,952 comments

Agent 2: Filter Agent

Role

The Filter Agent is the hard gate before the LLM. No post reaches Claude unless it passes every filter rule applied here. This agent exists because 64% of raw Reddit posts have deleted or removed bodies — a structural property of r/wallstreetbets where moderators routinely remove post content but preserve the thread. Sending deleted posts to Claude wastes API budget and distorts the sentiment signal.

Inputs

Input	Source	Description
df_posts_raw	Collector Agent	All raw posts, no cleaning applied
df_comments_raw	Collector Agent	All raw comments
min_score	config.py	Minimum upvote score threshold (default: 10)

Decision Logic

Three rules applied in sequence. A post must pass all three.

- **Rule 1 (Content availability):** Keep posts with a non-empty body OR at least one bundled comment. Replaced the original Phase 1 logic that removed all deleted posts. Raised retention from under 5% to 18.2%.

- **Rule 2 (Score threshold):** Remove posts with upvote score below 10. Filters fringe posts unlikely to reflect community sentiment.
- **Rule 3 (Stickied posts):** Remove stickied moderator announcements. No market signal.

After filtering, two derived columns are computed: `engagement_score = (score × 0.7) + (num_comments × 0.3)` and `llm_text` (title + body capped at 500 chars + top 5 comments by score).

Failure Handling

If the filtered DataFrame is empty, the Filter Agent returns an empty DataFrame. The main pipeline halts immediately, writes run_metadata.json with n_filtered=0, and returns. No API calls are made. This is stopping condition 1.

Outputs

Output	Description	Count (Phase 2 run)
df_filtered	Filtered posts with engagement_score and llm_text	283 posts (18.2% of 1,559 raw)

Agent 3: Sentiment Agent

Role

The Sentiment Agent is the LLM reasoning layer and the only agent that makes external API calls. It sends each filtered post's `llm_text` to `claude-sonnet-4-20250514` via the Anthropic Messages API and extracts 13 structured fields per post.

This agent exists because none of the tasks it handles can be done reliably by rule-based logic. In the Phase 2 run, 140 of 283 analyzed posts (49.5%) were flagged as sarcastic. A rule-based tool would have badly misclassified the large majority of these.

Inputs

Input	Source	Description
df_filtered	Filter Agent	Filtered posts with llm_text field
api_key	Environment variable (ANTHROPIC_API_KEY)	Anthropic API credentials
batch_size	config.py	Max posts to analyze per run (default: 500)

Decision Logic

For each post, the agent constructs a message with a static system prompt and the post's `llm_text` (capped at 800 characters). The system prompt is marked with `cache_control: ephemeral`, caching the prompt prefix after the first call and reducing input token costs by ~90% across the batch.

Claude is instructed to return valid JSON with no markdown wrapping. The prompt specifies sarcasm detection guidelines tailored to r/wallstreetbets conventions, ticker disambiguation rules (the $TICKER format is preferred; ambiguous short words such as AI, OPEN, and FAST require financial context), and rumour flagging criteria (named corporate events only). A 0.3-second sleep is applied between calls for rate limiting.

Failure Handling

- **JSONDecodeError:** Post receives neutral defaults (confidence=0.0). Retained in df_llm and identifiable by confidence=0.0.

- **Other exceptions:** Post returns None. Error counter increments. Pipeline continues. Error count written to run_metadata.json.

If the Sentiment Agent returns zero valid rows, the main pipeline halts. This is stopping condition 2.

Outputs and 13 LLM Fields

Field	Type	Description
sentiment	string	bullish | bearish | neutral
confidence	float	0.0–1.0 — Claude's confidence in its sentiment judgment
tickers	list	All stock tickers mentioned in the post
primary_ticker	string	Most important ticker, or UNKNOWN
emotions	list	greed, fear, hope, panic, euphoria, frustration, humor, sarcasm
post_type	string	DD | YOLO | loss | gain | news | question | meme | other
is_sarcastic	boolean	True if Claude determined the post is sarcastic
market_relevance	float	0.0–1.0 — how relevant the post is to market signal
key_insight	string	One-sentence summary of the post's market content
is_rumour	boolean	True if the post contains an unconfirmed corporate event claim
rumour_type	string	merger_speculation | acquisition_rumour | partnership_chatter | leadership_change | regulatory_decision | none
rumour_confidence	float	0.0–1.0 — Claude's confidence that this is a genuine rumour
rumour_summary	string	One-sentence description of the rumour, or none

Phase 2 run: 283 posts analyzed, avg confidence 0.56, 140 sarcastic posts (49.5%), 3 high-confidence rumours detected, 0 API errors.

Agent 4: Aggregator

Role

The Aggregator transforms per-post LLM output into two structured deliverables: a ticker-level sentiment summary (Track 1) and a separate rumour alert track (Track 2). It is a data processing agent with no external API calls. The two-track design is a core architectural decision — rumour alerts are kept separate because they serve a different purpose and require different disclaimers.

Inputs

Input	Source	Description
df_llm	Sentiment Agent	Per-post results with all 13 LLM fields
run_dir	Logger	Timestamped directory path for saving outputs
RUMOUR_THRESHOLD	config.py	Min rumour_confidence to surface an alert (default: 0.70)
LOW_CONF_THRESHOLD	config.py	Avg confidence below this triggers a warning flag (default: 0.60)
LOW_SAMPLE_THRESHOLD	config.py	Post count below this triggers a low-sample flag (default: 5)

Decision Logic

Groups posts by `primary_ticker`. Posts where `primary_ticker=UNKNOWN` are excluded from the ticker summary but remain in df_llm for trend analysis. Computes post count, bullish/bearish/neutral

percentages, average confidence, average relevance, sarcastic post count, and average engagement per ticker. Adds low_confidence and low_sample quality flags.

Rumour track: filters df_llm for `is_rumour=True` AND `rumour_confidence >= 0.70`. A monthly trend column is added to df_llm. All three outputs saved to the timestamped run directory before returning.

Failure Handling

No failure conditions halt the pipeline at this stage. If df_llm is empty, empty DataFrames are returned with a warning and the pipeline continues.

Outputs

Output	File	Description
ticker_summary	ticker_summary.csv	59 tickers in Phase 2 run. post_count, bullish/bearish/neutral %, avg_confidence, avg_relevance, sarcastic_count, avg_engagement, low_confidence flag, low_sample flag
rumour_alerts	rumour_alerts.csv	3 posts flagged in Phase 2 run
df_llm (modified)	sentiment_results.csv	All 283 analyzed posts with month column added

Agent 5: Output Agent

Role

The Output Agent is the human-facing layer. It receives the three outputs from the Aggregator and formats them into readable terminal reports. It does not re-analyze, re-score, or modify the data it receives. Its only job is presentation. This agent exists separately because presentation logic changes frequently and for different reasons than analysis logic. A display format change should not require touching the LLM prompt or the aggregation formula.

Inputs

Input	Source	Description
ticker_summary	Aggregator	Ticker-level sentiment DataFrame indexed by primary_ticker
rumour_alerts	Aggregator	High-confidence rumour posts (confidence >= 0.70)
df_llm	Aggregator	Per-post results with month column

Decision Logic

query_ticker(ticker): Renders sentiment bars, emotion breakdown, post type breakdown, monthly timeline, top 3 key insights with Reddit permalinks, and any matching rumour alerts. Triggers LOW CONFIDENCE warning if avg_confidence < 0.60, LOW SAMPLE warning if post_count < 5, and high sarcasm warning if sarcasm_rate > 10%.

run_trend_alerts(): Detects tickers where daily bullish or bearish percentage shifted by more than 15 percentage points over the past 7 days. Surfaces these as shift alerts.

compare_tickers(A, B): Side-by-side comparison of two tickers: sentiment summary, signal quality table, emotion breakdown, monthly timeline, and top 2 key insights per ticker.

All output captured by TeeLogger to both stdout and `trace.txt`. A `--replay` flag allows re-running the Output Agent against any saved run directory without API calls.

Failure Handling

Ticker not found in `ticker_summary`: prints “No data found” and lists available tickers. Empty `rumour_alerts`: rumour section silently omitted. Neither condition stops the pipeline.

Outputs

Output	Description
Terminal report	Three structured outputs per run: ticker reports (×3), trend alerts, ticker comparison
trace.txt	Full terminal output captured by TeeLogger
run_metadata.json	Config snapshot plus pipeline statistics

Section 3: Coordination Logic, Orchestration, and Error Handling

3.1 How the Pipeline Starts and Flows

The pipeline is invoked by running `main.py`. A timestamped run directory is created immediately on startup (e.g. `outputs/runs/run_20260412_110302`). A TeeLogger instance captures all stdout to both the terminal and `trace.txt` in real time.

Step	Agent	Receives	Passes Forward
1	Collector	File paths from config.py	df_posts_raw, df_comments_raw
2	Filter Agent	df_posts_raw, df_comments_raw	df_filtered (with engagement_score and llm_text)
3	Sentiment Agent	df_filtered, API key, batch_size	df_llm (13 LLM fields per post)
4	Aggregator	df_llm, run_dir	ticker_summary, rumour_alerts, df_llm (with month column)
5	Output Agent	ticker_summary, rumour_alerts, df_llm	Terminal report, trace.txt, run_metadata.json

3.2 The Filter Agent as Hard Gate

The Filter Agent is the only agent whose output determines whether any API costs are incurred. If no posts survive filtering, the pipeline halts before a single Claude call is made.

3.3 Two-Track Routing from the Aggregator

The Aggregator produces two independent outputs from the same input. Track 1 (ticker summaries) and Track 2 (rumour alerts) are passed separately to the Output Agent. The two tracks are not mutually exclusive: a post can be both a sentiment data point and a rumour alert.

3.4 Stopping Conditions

Condition	Where Checked	Trigger	What Happens
Stop 1	After Filter Agent	n_filtered == 0	Pipeline writes metadata, prints diagnostic, returns. No API calls.
Stop 2	After Sentiment Agent	n_analyzed == 0	Same behaviour as Stop 1.

3.5 Orchestration Approach: Plain Python with try/except

SentimentEdge manages agent handoffs and errors using plain Python orchestration: a single `main.py` script with explicit sequential function calls and `try/except` blocks placed at each agent boundary. Each agent is called directly as a Python function; its return value (a DataFrame or pair of DataFrames) is passed as an argument to the next agent. Error handling is co-located with the call site — if an agent raises an unexpected exception, the `except` block at that boundary logs the failure, writes `run_metadata.json` with the current counters, and halts. Row-level failures inside the Sentiment Agent are caught within a per-post `try/except` loop so that a single bad API response never propagates outward to the pipeline boundary. No orchestration framework (LangGraph, AutoGen, CrewAI) is used.

This approach is sufficient for three reasons. First, the pipeline is strictly sequential with no branching, parallelism, or dynamic agent selection — the overhead of a framework designed for concurrent or graph-based multi-agent systems would add complexity without enabling any capability this pipeline requires. Second, agent handoffs are simple Python function calls where the data contract is enforced by pandas column names; there is no message-passing protocol, no shared state store, and no retry queue that would justify a dedicated coordination layer. Third, all necessary logging and recovery behaviour is already implemented: TeeLogger captures the full execution record, stopping conditions write `run_metadata.json` before halting, and row-level errors in the Sentiment Agent are caught and counted without stopping the pipeline. For a five-agent sequential pipeline of this scope, plain Python with `try/except` provides the same guarantees as a framework with none of the abstraction cost.

3.6 Error and Partial-State Handling at Each Boundary

The pipeline is designed to continue past row-level failures wherever possible. The key design decision for the Sentiment Agent is: if the agent fails on a subset of posts — for example, 200 of 500 — the pipeline does **not** halt. Each failed post is skipped individually, the error counter increments, and the remaining successful posts are passed to the Aggregator. The pipeline only halts if *zero* valid results are returned (stopping condition 2). This means a partial batch is always preferred over an empty result, and the `errors` field in `run_metadata.json` reflects only actual per-post failures, not batch shortfall.

Agent Boundary	Error Type	Behavior	Partial State Preserved?
Collector → Filter	Missing JSONL file	FileNotFoundError raised with diagnostic message	No (run never starts)
Collector → Filter	Malformed JSONL lines	Lines skipped individually. Pipeline continues.	Yes
Filter → Sentiment	Zero posts after filtering	Pipeline halts. run_metadata.json written with n_filtered=0.	Yes
Sentiment → Aggregator	JSONDecodeError on single post	Neutral defaults returned (confidence=0.0). Post retained in df_llm. Error counter does not increment — the post is recoverable.	Yes
Sentiment → Aggregator	API exception on single post	Post skipped. Error counter increments. Pipeline continues with remaining posts. If 200 of 500 posts fail this way, the Aggregator receives 300 posts and runs normally.	Yes — 300 posts forwarded
Sentiment → Aggregator	Zero valid results (all posts fail)	Pipeline halts. Metadata written with n_analyzed=0.	Yes
Aggregator → Output	Empty df_llm input	Empty DataFrames returned with warning. Pipeline continues.	Yes
Output Agent	Ticker not found	Informational message printed. Other tickers unaffected.	Yes

Section 4: AI Usage Disclosure

Tools Used

Tool	Version	Who Used It	Purpose
Claude API (claude-sonnet-4-20250514)	Anthropic Messages API	Farrukh (pipeline)	Reddit post sentiment analysis, sarcasm detection, ticker extraction, and rumour identification in the Sentiment Agent
Claude Code (claude-sonnet-4-6)	CLI tool	Farrukh	Refactoring single-file prototype into modular directory structure; output agent rewrite and formatting; replay mode implementation; eval files and test cases
Claude.ai (claude-sonnet-4-6)	Web interface	Afaq	Architecture diagram, role definitions, coordination logic documentation, and Phase 2 document assembly

Claude API: Sentiment Agent

Each Reddit post is sent to `claude-sonnet-4-20250514` via the Anthropic Messages API. Claude returns a 13-field JSON object covering sentiment, confidence, tickers, emotions, sarcasm, market relevance, and rumour detection.

The system prompt instructs Claude to behave as a financial sentiment analyst specialising in Reddit finance communities. It specifies sarcasm detection guidelines tailored to r/wallstreetbets conventions, ticker disambiguation rules for ambiguous short words (AI, OPEN, FAST), and rumour flagging criteria requiring named corporate events. The full system prompt is in `agents/sentiment_agent.py` as the `_SYSTEM_PROMPT` constant.

Prompt caching is applied via `cache_control: ephemeral` on the system prompt, reducing input token costs by approximately 90% across the run.

What was changed manually after AI output:

- `JSONDecodeError` parse failures caught and replaced with neutral defaults (confidence 0.0)
- All 13 fields validated for presence using `.get()` with safe defaults
- Rumour alerts additionally filtered by `rumour_confidence >= 0.70` in the Aggregator

What was independently verified:

- Sarcasm detection rate manually reviewed (85% confirmed in prototype run)
- Rumour flagging manually reviewed — all 3 flagged posts in Phase 2 run were legitimate speculation threads
- Confidence scores for high-engagement meme posts confirmed low (DPZ: 0.20)

Claude Code: Refactor, Output Agent, and Eval

Used for:

- Refactoring the single-file Google Colab prototype into the modular directory structure
- Rewriting the Output Agent to produce three structured reports (Ticker Report, Trend Alerts, Ticker Comparison) with consistent formatting
- Implementing `--replay` mode (`python main.py --replay`) allowing the Output Agent to re-run against saved CSVs without API calls
- Fixing the misleading `errors` field in run_metadata.json (was `BATCH_SIZE - n_analyzed`; corrected to actual API failures only)

- Generating `eval/test_cases.md`, `eval/failure_log.md`, `eval/version_notes.md`, and `eval/case_outputs/`

What was changed manually: All agent logic verified against confirmed-working prototype behaviour. Config values cross-checked against Phase 1 thresholds. Eval test cases verified by running TC-06, TC-07, and TC-08 manually.

Claude.ai: Architecture and Documentation

Used for generating the Phase 2 architecture diagram, agent role definitions (Section 2), coordination logic (Section 3), and final document assembly. All statistics cross-checked against `run_metadata.json` and `sentiment_results.csv` from the actual Phase 2 run.

Section 5: Team Contribution Update

Phase 2 Contributions by Section

Team Member	Phase 2 Contribution
Farrukh	Full pipeline implementation: Collector, Filter, Sentiment, Aggregator, and Output agents. Modular refactor from Colab prototype. Prompt caching on Sentiment Agent. Run logging system (TeeLogger, trace.txt, run_metadata.json). Output agent rewrite with structured three-report format. Replay mode (--replay flag). Bug fixes F-01 through F-05. Batch size tuned to 500. 1-week prototype run on real dataset. Eval files: test_cases.md (TC-01 through TC-08 with evidence), failure_log.md, version_notes.md, case_outputs/. Section 1 and Appendix.
Pablo	Evaluation plan: 8 test scenarios with success criteria and measurement methods. Sarcasm ground-truth label set and annotation protocol (inter-annotator agreement via Krippendorff's alpha). Tools, memory, and state design section. Stateless design justification. Updated risk section with concrete mitigations per failure mode.
Moid	Full dataset download and distribution to team.
Afaq	Architecture diagram (all 5 agents, I/O, handoffs, stopping conditions, two output tracks, human-in-the-loop). Agent role definitions (Section 2). Coordination logic, orchestration justification, and error handling specification (Section 3). AI usage disclosure (Section 4). Contribution table (this section). Final document assembly.

Effort Estimates

Team Member	Phase 1	Phase 2	Total
Farrukh	~5 hrs	~20 hrs	~25 hrs
Pablo	~4 hrs	~4 hrs	~8 hrs
Moid	~2 hrs	~3 hrs	~5 hrs
Afaq	~3 hrs	~7 hrs	~10 hrs

Updated Role Summary

Component	Phase 1 Lead	Phase 2 Lead
Data pipeline: Collector and Filter agents	Farrukh	Farrukh
LLM integration: Sentiment Agent	Farrukh	Farrukh
Aggregation logic: Aggregator Agent	Farrukh	Farrukh
Output and reporting: Output Agent	Farrukh	Farrukh
Evaluation design and test cases	Pablo	Pablo + Farrukh
Phase documentation and diagrams	Afaq	Afaq
Final report assembly	Afaq	Afaq
Dataset acquisition	Not applicable	Moid

Section 6: Tools, Memory, and State Design

SentimentEdge uses five agents, each with a clearly scoped set of tools, a defined read/write boundary, and a rationale for why that boundary was drawn where it is.

Agent	Tools / APIs	Reads	Writes	State Boundary	Why This Design
Collector	Local filesystem, JSONL, pandas	Raw post and comment JSONL files	Raw DataFrames	Ingestion only	Keeps data access narrow and platform-agnostic so future source changes stay isolated to one module
Filter	pandas, config.py, comment bundler utility	Raw DataFrames from Collector	Filtered DataFrame with bundled comments, engagement score, and llm_text	Deterministic preprocessing only	Keeps cleanup, scoring, and deleted-post recovery outside the LLM for consistency and lower cost
Sentiment	Anthropic Messages API, JSON parser, pandas	Filtered DataFrame	df_llm with per-post structured labels	External reasoning boundary	Centralises ambiguous language understanding in one place while containing parse fallback and row-level API errors locally
Aggregator	pandas, CSV writer	df_llm	sentiment_results.csv, ticker_summary.csv, rumour_alerts.csv	Synthesis and routing only	Separates statistical rollups and rumour routing from rendering so output logic stays simple

Agent	Tools / APIs	Reads	Writes	State Boundary	Why This Design
Output	pandas, terminal formatting helpers	Aggregated DataFrames or replay CSVs	Terminal reports only	Presentation only	Keeps the human-facing layer lightweight and replayable without repeating upstream processing or API calls

State Design

The data path is intentionally simple: JSONL on disk → raw pandas DataFrames → filtered DataFrame with bundled comments and `llm_text` → Claude JSON response per post → parsed dicts collected into `df_llm` → aggregated DataFrames → per-run CSV and JSON artifacts.

There is no cross-session persistence layer, vector store, long-term thread memory, or database. State exists only for the lifetime of a run in memory, plus the saved run artifacts under `outputs/runs/`. That stateless design was chosen for four reasons:

- The project scope is small enough that persistent storage would add complexity without enabling a core capability.
- Per-run artifacts are easier to reproduce and debug than stateful systems with hidden carryover.
- Avoiding persistent memory reduces governance overhead.
- The project analyses time-sensitive market chatter, so stale stored state is more likely to confuse later runs than to help them.

Section 7: Evaluation Plan and Test Cases

`All cases run against: outputs/runs/run_20260412_110302/`

Dataset: r/wallstreetbets | 1,559 raw posts | 283 analyzed | 59 tickers | 3 rumours flagged

Case ID	Type	Input / Scenario	Expected Behaviour	Actual Behaviour	Outcome	Evidence
TC-01	Happy path	Replay saved run. Query SPY — top ticker, 35 posts, low_sample=False.	Full report renders across all 6 sections. No crash. No missing fields.	SPY report rendered correctly: sentiment bars, emotion breakdown (11 categories), post type breakdown (7 types), monthly timeline, 3 top insights with Reddit permalinks, 2 rumour alerts with disclaimer. No crash.	PASS	eval/case_outputs/TC-01_SPY_report.txt; ticker_summary.csv — SPY row: post_count=35, low_sample=False
TC-02	Sarcasm handling	Inspect SPY and TSLA reports. Both have sarcasm_rate > 10%.	High sarcasm warning displayed. [SARCASTIC] label in insights. Sarcastic posts have lower confidence than non-sarcastic.	SPY: 14/35 sarcastic (40%) — warning triggered. TSLA: 4/8 sarcastic (50%) — warning triggered. [SARCASTIC] labels present. Sarcastic meme post	PASS	eval/case_outputs/TC-02_sarcasm_warnings.txt; ticker_summary.csv — sarcastic_count: SPY=14, TSLA=4

Case ID	Type	Input / Scenario	Expected Behaviour	Actual Behaviour	Outcome	Evidence
				conf=0.30 vs DD post conf=0.70.		
TC-03	Low confidence warning	Inspect TSLA report. avg_confidence=0.51, below LOW_CONF_THRESHOLD=0.60.	LOW CONFIDENCE warning at report header. Full report still renders.	TSLA report displays "⚠ LOW CONFIDENCE" at header. All sections rendered. avg_confidence=0.51 confirmed in ticker_summary.csv.	PASS	eval/case_outputs/TC-03_TSLA_low_confidence.txt; ticker_summary.csv — TSLA: avg_confidence=0.51, low_confidence=True
TC-04	Viral meme low signal	Inspect DPZ entry from ticker_summary. Highest engagement (9,656.8) in dataset.	market_relevance < 0.4 and confidence < 0.5. low_confidence=True.	DPZ: avg_confidence=0.20, avg_relevance=0.10, low_confidence=True. Meme post about Domino's pizza orders near the Pentagon — correctly identified as no market signal despite highest engagement.	PASS	eval/case_outputs/TC-04_DPZ_low_signal.txt; ticker_summary.csv — DPZ row
TC-05	Rumour detection	Inspect rumour_alerts.csv and SPY report.	Flagged posts appear in rumour_alerts.csv. Rumour Alerts section in ticker report with disclaimer.	3 posts flagged: Iran ceasefire/oil (conf=0.80), Iran missile salvo (conf=0.70), OpenAI/Anthropic/SpaceX IPO index bypass (conf=0.70). All in rumour_alerts.csv. SPY report shows Rumour Alerts section with source links and disclaimer.	PASS	eval/case_outputs/TC-05_rumour_alerts.txt; rumour_alerts.csv — 3 rows
TC-06	Replay mode	python main.py --replay with no argument.	Skips agents 1–4. Loads CSVs. Renders all reports. No API calls. Completes in seconds.	Auto-selected most recent run. All three reports rendered. emotions/tickers columns restored via ast.literal_eval. Completed in under 5 seconds. No API key required.	PASS	eval/case_outputs/TC-06_replay_mode.txt; main.py replay() function
TC-07	Ticker not found	Call query_ticker('XYZ', ...) against saved outputs. XYZ not in ticker_summary.	"No data found for $XYZ" message. Available tickers listed. No crash.	Output Agent printed "No data found for $XYZ." and listed all 59 available tickers. No KeyError. Pipeline unaffected.	PASS	eval/case_outputs/TC-07_query_miss.txt; agents/output_agent.py — ticker not found guard
TC-08	Filter zero result	Set MIN_POST_SCORE=100000. Run full pipeline.	Pipeline stops after Filter Agent. run_metadata.json written with n_filtered=0. No API calls.	Pipeline halted after Agent 2. "✗ Pipeline stopped — no posts survived filtering." run_metadata.json: n_filtered=0, n_analyzed=0, errors=0. No Agent 3 header in trace.txt.	PASS	eval/case_outputs/TC-08_filter_zero_result.txt; main.py lines 64–72

Success Criteria Results

Criterion	Target	Result	Status
Sentiment accuracy	Avg confidence ≥ 0.75 on high-quality posts	SPY (35 posts): avg_confidence=0.67	Fail
Sarcasm detection	Warning triggers above 10% rate	SPY 40%, TSLA 50% — both triggered correctly	Pass
Rumour detection	Posts routed to separate track with disclaimer	3 posts flagged, all in rumour_alerts.csv	Pass
Ticker extraction	No false positives on ambiguous tokens	No AI, OPEN, FAST in ticker_summary	Pass
Pipeline throughput	500 posts processed under 10 min	283 posts in 1,113s (18.5 min)	Fail
Replay mode	Output reruns without API key	Confirmed — under 5 seconds	Pass
Filter quality	≥ 30% retention	283/1,559 = 18.2%	Fail
Coverage	≥ 10 tickers with ≥ 5 posts	3 tickers with ≥ 5 posts (SPY, TSLA, TACO)	Fail
Failure handling	Zero-filter and query-miss handled gracefully	TC-07 and TC-08 confirmed	Pass
Viral meme handling	High-engagement low-signal post correctly scored	DPZ: conf=0.20, relevance=0.10	Pass

Notes on Failing Criteria

Filter quality (18.2%): The 64% deleted-post problem is mitigated but not eliminated. Rule 1 already raises retention from under 5% to 18.2%. Further improvement requires lowering MIN_POST_SCORE below 10, which risks including fringe posts.

Confidence (0.67 for SPY): The April 2026 market downturn produced a 49.5% sarcasm rate. Heavy irony naturally depresses confidence scores. This is a dataset characteristic documented in F-03, not a pipeline failure.

Pipeline throughput (18.5 min): Only 283 posts were processed (fewer passed filtering than the 500 batch cap). `errors: 217` previously shown in run_metadata.json was a misleading label — it reflected BATCH_SIZE(500) − n_analyzed(283) = 217, not actual API errors. This has been corrected: errors now reflects posts sent to Claude that returned None. In the Phase 2 run, errors = 0.

Coverage (3 tickers ≥ 5 posts): Direct consequence of low filter retention. 283 posts across 59 tickers yields an average of 4.8 posts per ticker. A larger or multi-week dataset would improve this significantly.

7.1 Sarcasm Ground-Truth Benchmark

Dataset and sampling. 50 posts were drawn from the Phase 2 run of 283 analyzed posts across three stratified sample buckets: 20 posts the model labeled sarcastic (`model_sarcastic`), 20 posts the model labeled not sarcastic (`model_not_sarcastic`), and 10 posts identified as borderline or mixed-signal (`borderline_mixed`). Stratified sampling was used so that the benchmark covers the model's confident predictions in both directions as well as the ambiguous cases where disagreement is most likely.

Who annotates. Pablo completed the primary annotation pass for all 50 posts. A second annotator (teammate_1) completed an independent pass on each post without access to Pablo's labels. A third adjudicated label was produced for any post where the two annotators disagreed, resolving to the label that better matched the documented cue type and financial context.

Annotation schema. Each post was labeled using the following fields:

Field	Values	Description
annotation_label	sarcastic / not_sarcastic / uncertain	Primary sarcasm judgment
annotation_confidence	high / medium / low	Annotator's confidence in the label
cue_type	reversal / irony / hyperbole / meme_joke / unclear	Type of sarcasm cue present, if any
financial_context_present	yes / no	Whether the post contains a genuine financial claim alongside the sarcasm
annotation_rationale	free text	One-sentence explanation of why the label was assigned
adjudicated_label	sarcastic / not_sarcastic / uncertain	Final resolved label after second-pass review

Annotation guidelines. A post is labeled `sarcastic` if the intended meaning is the opposite of or substantially different from the literal meaning of the text — for example, a post framing a bathroom joke as geopolitical market analysis, or using the phrase "Everything is priced in" to mock efficient-market theory. A post is labeled `not_sarcastic` if the primary claim is literal, even if the tone includes WSB-style profanity or self-deprecating humor. A post is labeled `uncertain` if the sarcasm lives in comments rather than the post itself, or if the dominant tone is ambiguous at the post level (e.g., daily discussion threads that mix sincere and ironic replies). `Uncertain` posts are excluded from the precision/recall calculation.

Inter-annotator agreement. Agreement is measured using Krippendorff's alpha on the three-class label set (sarcastic / not_sarcastic / uncertain). The minimum acceptable threshold is $\alpha \geq 0.67$. The target threshold is $\alpha \geq 0.80$. Posts labeled `uncertain` by either annotator are included in the alpha calculation to penalise systematic avoidance of hard cases. If alpha falls below 0.67, the annotation guidelines are revised and the borderline bucket is re-annotated before any benchmark claims are reported.

Benchmark metric. Once adjudicated labels are finalised, model precision and recall are computed on the `sarcastic` vs. `not_sarcastic` subset (excluding `uncertain` posts). The benchmark result is reported as: precision, recall, F1, and the count of posts excluded as uncertain.

Section 8: Risk and Governance

Failure / Misuse	Why It Matters	Control	Residual Risk	Evidence
Deleted or removed post bodies	Large portions of WSB data lose body text, collapsing coverage if handled too aggressively	Keep posts with body OR bundled comments; log filtered counts per run	Title-only posts can still be noisy	trace.txt, run_metadata.json, filter logic in code
Ambiguous ticker extraction	Tokens such as OPEN or AI can be ordinary language rather than equities	Prompt explicitly requires financial context for ambiguous short tokens; evaluation includes manual spot-checks	Some borderline uses still require human review	sentiment_results.csv, eval/test_cases.md

Failure / Misuse	Why It Matters	Control	Residual Risk	Evidence
Sarcasm misclassification	WSB language often inverts literal sentiment	Dedicated sarcasm evaluation set, explicit annotation protocol, agreement threshold before final benchmark claims	Thread-level sarcasm remains hard when irony lives in comments	eval/sarcasm_ground_truth_protocol.md (Pablo)
Viral meme overpowering signal	High-engagement memes can dominate selection despite weak market content	Use market_relevance, confidence, and post_type as separate signal-quality indicators	Aggregation does not directly down-weight low-relevance posts	sentiment_results.csv, ticker_summary.csv, eval/failure_log.md
Rumour false positives	False corporate-event rumours can appear actionable	Keep rumours in a separate output track, require confidence threshold, label as unverified	High-confidence false positives are still possible	rumour_alerts.csv, output disclaimer text
Partial API or parse failures	LLM output failures can silently distort sample counts	Neutral defaults for JSON parse failures; continue on row-level API errors; log errors	Neutral defaults can lower confidence averages	trace.txt, sentiment_results.csv, run_metadata.json
Market manipulation allegation misuse	Users may read community allegations as established fact	Treat manipulation claims as allegations only	Users may still over-interpret emotionally charged posts	Report language, output disclaimer
Rumour alert misuse	A separate rumour track can be mistaken for a trading signal feed	Keep rumours separate from ticker summaries; preserve “unverified social media speculation” disclaimer	Users may still act on rumours despite the warning	rumour_alerts.csv, output agent logic
Financial advice liability	A market summary can be mistaken for personalised advice	Keep outputs descriptive, source-linked, non-personalised, and explicitly non-prescriptive	Some users may still treat summaries as recommendations	Footer disclaimer in trace, report language
Stale time-bounded outputs	Old social chatter can be read as current market guidance	Stamp outputs with run date and latest data date	Screenshots can circulate without context	trace.txt, run_metadata.json, output footer

Appendix: Prototype Evidence

A.1 Pipeline Execution Trace (excerpt)

```
SentimentEdge — Run started at 2026-04-12 11:03:02
Run directory: outputs/runs/run_20260412_110302
Batch size:    500 posts
Model:         claude-sonnet-4-20250514 (prompt caching enabled)

=========================================================
AGENT 1 — COLLECTOR
=========================================================
```

```
Loading posts...
  ✓ Loaded 1559 posts
Loading comments...
  ✓ Loaded 224952 comments
✓ Collector complete
   Posts loaded:    1559
   Comments loaded: 224952

=========================================================
AGENT 2 — FILTER AGENT
=========================================================
Input: 1559 posts
  Bundled comments for 436 unique posts
  Posts with body content:      396
  Posts with bundled comments:  398
  Posts with body OR comments:  568
  Posts removed by score (<10): 284
  After score filter:           284
  After stickied-post removal:  283
✓ Filter Agent complete
   Input: 1559 | Output: 283 | Removed: 1276 | Retention: 18.2%

=========================================================
AGENT 3 — SENTIMENT AGENT
=========================================================
  Posts selected:   283
  Prompt caching:   enabled
  Progress: 10/283 | Elapsed: 40s | ETA: 1088s | Errors: 0
  ...
  Progress: 280/283 | Elapsed: 1102s | ETA: 12s | Errors: 0
✓ Sentiment Agent complete
   Processed: 283 | Errors: 0 | Time: 1113s
   Sentiment: bearish 119 | bullish 100 | neutral 64
   Avg confidence: 0.56 | Sarcastic: 140 | Rumours: 29 (raw flags)

=========================================================
AGENT 4 — AGGREGATOR
=========================================================
  Ticker summary (59 tickers) — top 5 by post count:
  SPY:  35 posts | 31.4% bull | 57.1% bear | conf: 0.67
  TSLA:  8 posts | 12.5% bull | 50.0% bear | conf: 0.51
  TACO:  5 posts | 60.0% bull | 20.0% bear | conf: 0.62
  QQQ:   4 posts | 50.0% bull | 50.0% bear | conf: 0.78
  META:  4 posts | 25.0% bull | 50.0% bear | conf: 0.75
  Rumour alerts (confidence >= 0.7): 3
✓ Aggregator complete — Tickers: 59 | Rumours flagged: 3

[Output Agent reports follow...]
Total pipeline time: 1120.0s
All outputs saved to: outputs/runs/run_20260412_110302
```

A.2 run_metadata.json


```
{
  "run_timestamp": "2026-04-12T11:21:42.866866",
  "run_dir": "outputs/runs/run_20260412_110302",
  "config": {
    "batch_size": 500,
    "min_post_score": 10,
    "rumour_threshold": 0.7,
    "low_conf_threshold": 0.6,
    "top_n_comments": 5,
    "shift_threshold": 15,
    "min_posts_per_day": 3
  },
  "pipeline_duration_seconds": 1120.0,
  "n_posts_raw": 1559,
  "n_comments_raw": 224952,
  "n_filtered": 283,
  "n_analyzed": 283,
  "errors": 0,
  "tickers_found": 59,
  "rumours_flagged": 3,
  "avg_confidence": 0.56,
  "sarcastic_count": 140
}
```


A.3 sentiment_results.csv — Sample Rows (5 of 283)

post_id	title	sentiment	confidence	primary_ticker	is_sarcastic	market_relevance	post_type	key_insight
1sfw2hs	Iran closes Strait of Hormuz again...	bearish	0.70	UNKNOWN	False	0.90	news	Geopolitical escalation in Middle East driving bearish sentiment on broad market
1sf7tk8	The dominos index closest to The Pentagon just spiked...	neutral	0.20	DPZ	True	0.10	meme	Satirical post about Domino's pizza orders near Pentagon with no real market implications
1se0z9t	JPMorgan warns Tesla stock could sink 60% in new note.	bearish	0.85	TSLA	False	0.90	news	JPMorgan issues strong sell rating on Tesla with 60% downside target
1scfwlv	Cash gang watching	neutral	0.30	SPY	True	0.40	meme	Meme post joking about cash holders

post_id	title	sentiment	confidence	primary_ticker	is_sarcastic	market_relevance	post_type	key_insight
	the news alongside the SPY chart							waiting for market dips while markets continue rising
1sfbxx0	Oil Prices and News — U.S. oil prices plunge 16%...	bullish	0.70	UNKNOWN	False	0.90	news	Oil prices dropped 16% below $100 due to Trump-Iran ceasefire news

A.4 ticker_summary.csv — Top 5 Tickers by Post Count

Ticker	Posts	Bullish %	Bearish %	Neutral %	Avg Conf	Avg Rel	Sarcastic	Avg Engage	Low Conf	Low Sample
SPY	35	31.4	57.1	11.4	0.67	0.73	14	665.3	False	False
TSLA	8	12.5	50.0	37.5	0.51	0.50	4	1031.1	True	False
TACO	5	60.0	20.0	20.0	0.62	0.60	2	69.5	False	False
QQQ	4	50.0	50.0	0.0	0.78	0.80	1	340.8	False	True
META	4	25.0	50.0	25.0	0.75	0.85	1	287.8	False	True